1. Header

Project Name: Community Intelligence Dashboard
Sprint: Sprint 3 – Engagement Loop
Story #10: Answer Notification System
Priority: Medium (Completes Engagement Loop)
Owner: Community Intelligence Team
User Role: Community Member

User Story:
As a user, I want to receive notifications when someone answers a question I asked that was previously flagged as unanswered so that I don't miss the response.

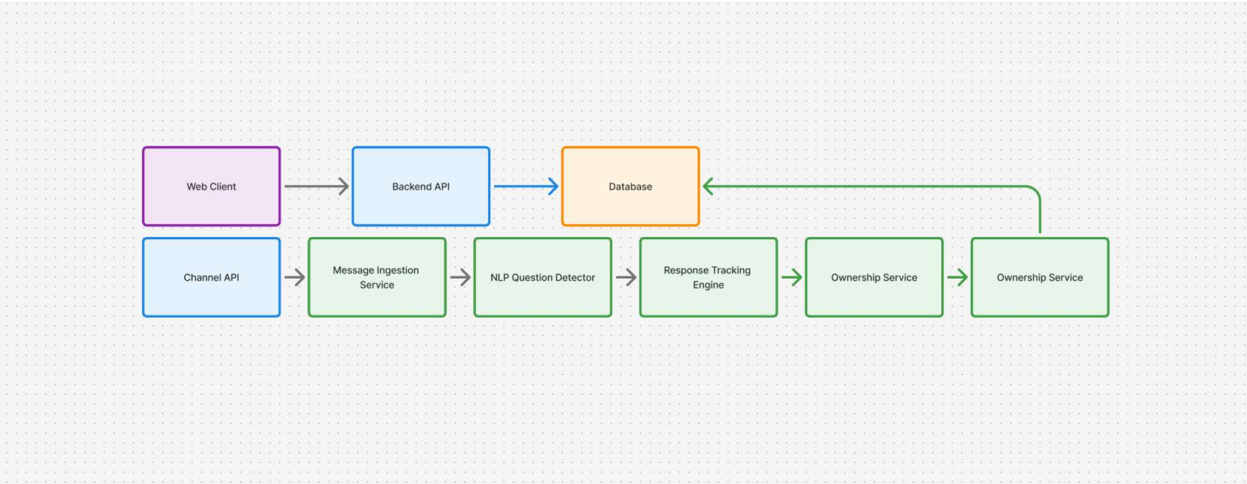
Sprint Goal:
Notify question-askers automatically when their previously unanswered question is marked as answered.

Dependency:
This sprint builds directly on Sprint 2 (Unanswered Question Detection), specifically:

- QuestionDetectionService
- ResponseTrackingService
- Question State Machine
- questions database table
- isAnswered flag logic

---

2. Architecture Diagram



Component Placement

Existing (From Sprint 2 – Server)

- Message Ingestion Service
- Question Detection Service
- Response Tracking Service
- PostgreSQL Database

These components already detect when a question transitions from *Pending* → *Answered*.

New Components (Sprint 3)

Server Layer

- Ownership Service
- Answer Detection Listener (Event Trigger)
- Notification Service

Notification Channels

- In-App Notification System
- Email Service
- Push Notification Service

Client Layer

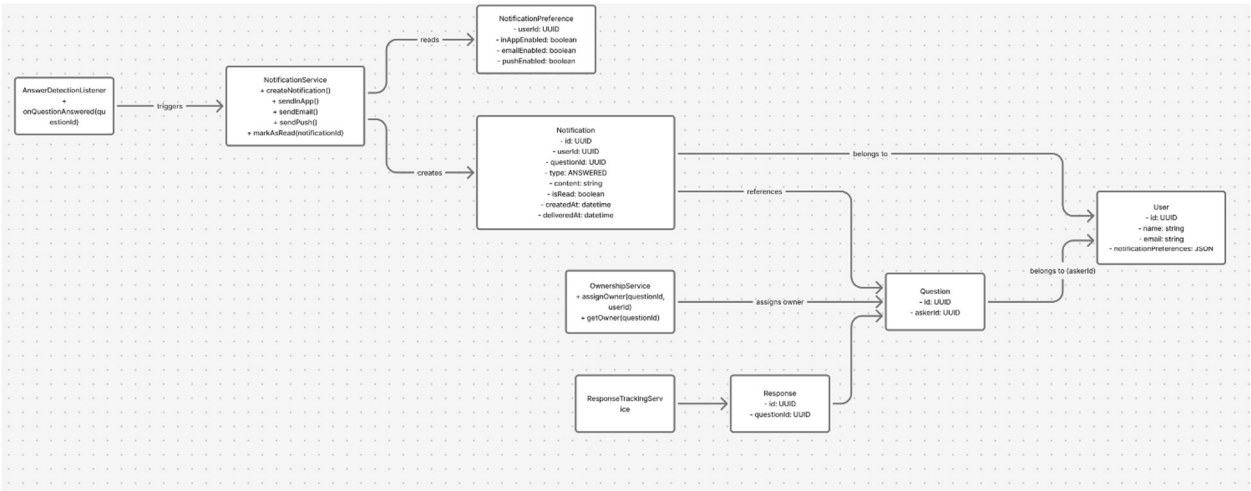
- Web Dashboard / Application
    - Notification Bell
    - Toast Notifications
    - Notification History Panel

Information Flow

1. ResponseTrackingService updates Question.isAnswered = true
2. Database emits event or trigger
3. AnswerDetectionListener captures event
4. OwnershipService retrieves question asker (askerId)
5. NotificationService creates notification record
6. Notification sent via enabled channels
7. User receives notification
8. User opens notification → status marked as Read

---

3. Class Diagram



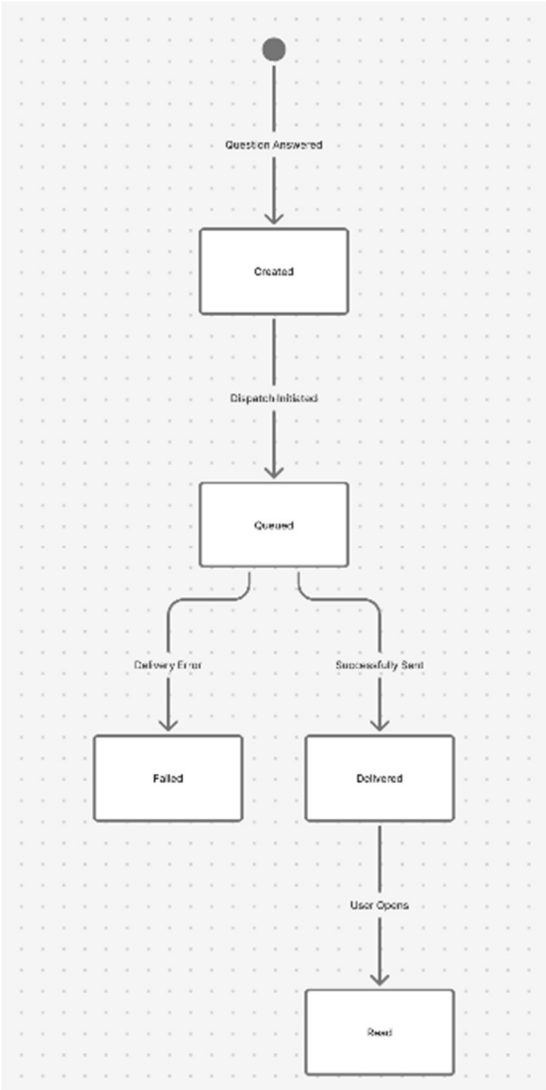
---

4. List of Classes

- User
- Notification
- NotificationPreference
- OwnershipService
- AnswerDetectionListener
- NotificationService
- Question (from Sprint 2)
- Response (from Sprint 2)
- ResponseTrackingService (from Sprint 2)

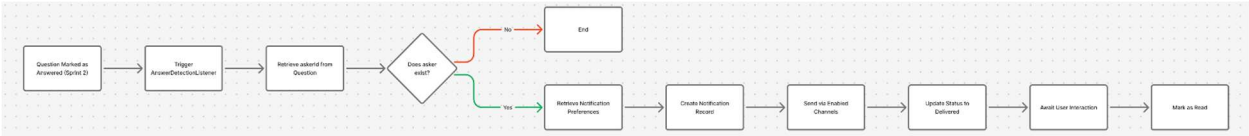
---

5. State Diagrams



---

6. Flow Chart



---

7. Development Risks and Failures

1. Duplicate Notifications

If answer status toggles repeatedly.
Mitigation: Ensure idempotent event handling.

2. Missed Events

If event listener fails.
Mitigation: Retry mechanism and event queue persistence.

3. Notification Fatigue

Too many notifications reduce engagement.
Mitigation: User-configurable preferences.

4. Email Delivery Failures

Mitigation: Retry logic and delivery tracking.

5. Race Conditions

Multiple responses trigger multiple events.
Mitigation: Only notify on first transition to Answered.

---

8. Technology Stack

Frontend

- React.js
- TypeScript

Backend

- Node.js (Express) or Python (FastAPI)
- Event-driven architecture

Database

- PostgreSQL

Notification Infrastructure

- In-app via DB + WebSocket
- Email via SendGrid or similar
- Push via Firebase Cloud Messaging

Infrastructure

- Docker
- Cloud Provider (AWS/GCP/Azure)

---

9. APIs

GET /api/notifications

Returns all notifications for authenticated user.

Response:

```
[
  {
    "id": "uuid",
    "type": "ANSWERED",
    "content": "Your question has been answered.",
    "isRead": false,
    "createdAt": "2026-02-14T15:00:00Z"
  }
]
```

POST /api/notifications/read

Marks notification as read.

Internal Event

onQuestionAnswered(questionId: UUID)

Triggers notification workflow.

---

10. Public Interfaces

User Notification Panel

Features:

- Notification Bell Icon
- Real-time updates
- Mark as read
- Notification history
- Click notification → Navigate to answer

---

11. Data Schemas

Updated questions table (Sprint 3 addition)

Add:

Column	Type
asker_id	UUID

---

notifications table

Column	Type
id	UUID
user_id	UUID
question_id	UUID
type	VARCHAR
content	TEXT
is_read	BOOLEAN

Column	Type
created_at	TIMESTAMP
delivered_at	TIMESTAMP

---

notification_preferences table

Column	Type
user_id	UUID
in_app_enabled	BOOLEAN
email_enabled	BOOLEAN
push_enabled	BOOLEAN

---

12. Security and Privacy

Authentication

- JWT-based authentication
- Only authenticated users can access notifications

Authorization

- Users can only view their own notifications

Data Protection

- Encrypted in transit (HTTPS)
- Encrypted at rest

Privacy

- No external sharing of question content
- Email notifications contain minimal data

---

13. Risks to Completion

Technical Risks

- Event-driven system not fully stable
- WebSocket integration complexity

Time Risks

- Multi-channel notification implementation may exceed sprint scope

Dependency Risks

- Sprint 2 answer detection must function correctly
- Database schema migration required